



uGDX School of Technology

Mini Project Report on

“VaaniSetu: An AI-Powered Multilingual Document Reader using Amazon Bedrock, Amazon Polly, Amazon Rekognition and AWS Lambda”

Submitted in partial fulfillment of the requirements
for the course

Course: Cloud Application Development (CAP 303)

Class: TY B. Tech (Semester V)

By

Student Name1	App ID
Student Name2	App ID
Student Name3	App ID

Supervisor:

Dr. Kunal Meher
(Assistant Professor)

AY: 2026-2027



uGDX School of Technology

CERTIFICATE

This is to certify that the project entitled “**VaaniSetu: An AI-Powered Multilingual Document Reader using Amazon Bedrock, Amazon Polly, Amazon Rekognition and AWS Lambda**” is a bonafide work of

Student Name1	App ID
Student Name2	App ID
Student Name3	App ID

submitted in partial fulfilment of the requirement for the course “**Cloud Application Development**” of Third Year of Bachelor degree in Technology (CS, AI & ML) during academic year 2026-27.

Supervisor/Guide
(Dr. Kunal Meher)

External Examiner

Table of Contents

- 1. Introduction**
- 2. Block Diagram**
- 3. Technologies used**
 - a. Development Technologies**
 - b. Cloud Services (Deployment)**
- 4. Code**
- 5. Screenshots**
- 6. Conclusion**

1. Introduction

Everyday life in India runs on paper: doctors' prescriptions, electricity bills, college notices, bank letters and government forms. Many people cannot read these documents comfortably: people with low vision, elderly citizens, people with limited literacy, and migrants who read a different language from the one the document is printed in. Missing a due date on a bill or misreading a dosage on a prescription has real consequences.

VaaniSetu (Hindi for "bridge of voice") is a serverless cloud application that solves this problem. The user simply photographs a document with a phone or uploads an image. The application reads the document, understands what it means, explains it in simple words in the language the user chooses, and **speaks the explanation aloud**. It also lists the key facts and the actions the reader must take, such as paying a bill before a deadline.

1.1 Objectives

- Build a fully serverless web application on AWS that needs no servers to manage.
- Combine at least three cloud services: this project integrates **eight core AWS services** (S3, Lambda, API Gateway, Rekognition, Bedrock, Polly, DynamoDB, Amplify) plus IAM and CloudWatch.
- Use generative AI (Amazon Nova on Amazon Bedrock) to read and explain documents in seven languages, and neural text-to-speech (Amazon Polly) to speak the explanation.
- Keep the system secure (private storage, pre-signed URLs, least-privilege IAM) and cost-efficient (pay-per-use, no idle cost).

1.2 Key Features

- **Photo to speech in about 10 seconds:** upload or capture a document photo and hear it explained.
- **Seven output languages:** English, Hindi, Spanish, French, German, Japanese and Arabic, each with a natural Amazon Polly neural voice. The source document can be in any language (for example, a Hindi bill explained in English).
- **Structured understanding:** document type, title, plain-language summary, key points and action items with deadlines, plus the full transcribed text.
- **Smart gatekeeping:** Amazon Rekognition checks that the photo really contains text before the more expensive AI model is called.
- **History:** every processed document is stored in DynamoDB and can be replayed.
- **Accessible design:** large type, high contrast, keyboard navigation and a live progress tracker that shows each AWS service as it runs.

2. Block Diagram

Figure 2.1 shows the architecture of VaaniSetu. All components run in the AWS Asia Pacific (Mumbai) region, ap-south-1.

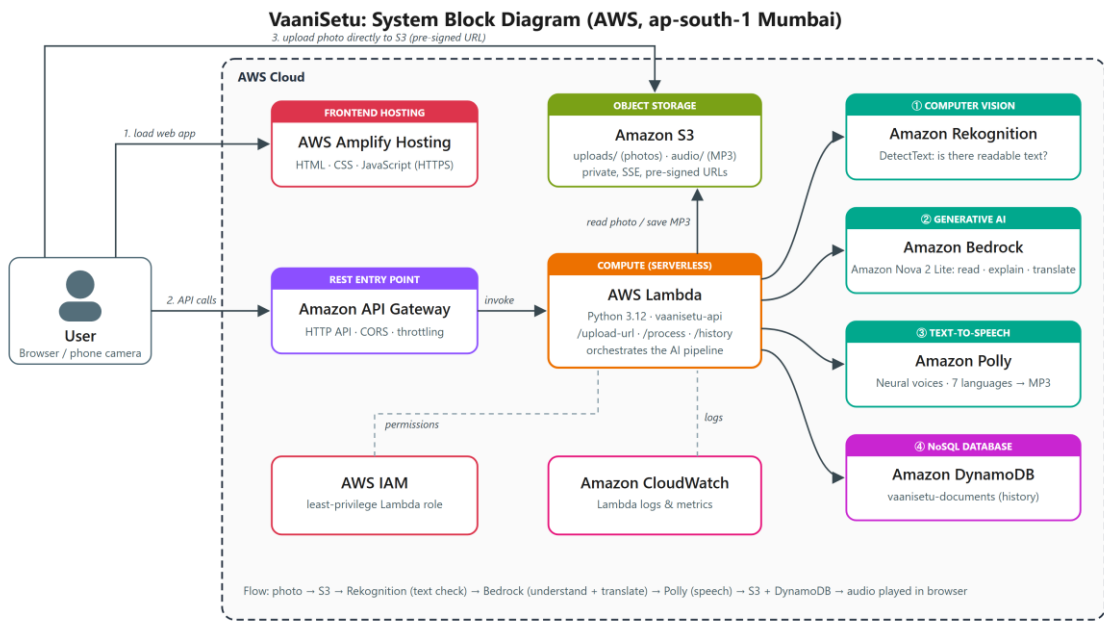


Figure 2.1: System block diagram of VaaniSetu on AWS

2.1 Workflow

1. The user opens the web application, which is served over HTTPS by **AWS Amplify Hosting**.
2. The browser resizes the photo and calls **POST /upload-url** on **Amazon API Gateway**. **AWS Lambda** returns a short-lived pre-signed URL.
3. The browser uploads the photo **directly to a private Amazon S3 bucket** using that URL, so large files never pass through the API.
4. The browser calls **POST /process**. Lambda asks **Amazon Rekognition** (DetectText) whether the image contains readable text. Photos without text are rejected at this point.
5. Lambda sends the image to **Amazon Nova 2 Lite on Amazon Bedrock**, which transcribes the document and returns JSON: document type, title, a spoken summary in the chosen language, key points and action items.
6. **Amazon Polly** converts the summary to speech with a neural voice. The MP3 file is saved to S3.
7. The result is stored in **Amazon DynamoDB** and returned to the browser with pre-signed links to the image and audio, and the audio plays automatically.
8. **AWS IAM** limits the Lambda function to exactly these actions, and **Amazon CloudWatch** records logs and metrics for every request.

3. Technologies Used

3.1 Development Technologies

Technology	Purpose in VaaniSetu
Python 3.12	Backend business logic running on AWS Lambda (arm64 / Graviton).
Boto3 (AWS SDK for Python)	Calls S3, Rekognition, Bedrock (Converse API), Polly and DynamoDB from Lambda.
HTML5	Page structure: upload area, language selector, results, history and "how it works" sections, with semantic, accessible markup.
CSS3	Styling: responsive grid layout, custom design tokens, animated pipeline tracker and waveform, mobile layout and reduced-motion support.
JavaScript (ES2020)	Dynamic features: drag-and-drop and camera upload, in-browser image resizing (Canvas API), Fetch API calls, live pipeline status, audio playback and history cards.
JSON	Data format between browser, API and AI model, and for IAM policies.
AWS CLI and Bash	Infrastructure provisioning and repeatable deployment (infra/deploy.sh).
Git and GitHub	Version control with small, atomic commits.

3.2 Cloud Services (Deployment)

The following AWS services are used. All of them are serverless or fully managed, so the application has no servers to patch and costs nothing while idle.

AWS Service	Role in VaaniSetu	Configuration
Amazon S3	Stores uploaded document photos (uploads/) and generated speech (audio/).	Private bucket, public access blocked, SSE-S3 encryption, CORS for browser upload, 30-day lifecycle on uploads, pre-signed URLs.
AWS Lambda	Serverless backend that runs the whole AI pipeline.	vaanisetu-api, Python 3.12, arm64, 512 MB, 29 s timeout.
Amazon API Gateway	Public REST entry point for the web app.	HTTP API with routes POST /upload-url, POST /process, GET /history; CORS; throttling at 10 req/s (burst 20).
Amazon Bedrock	Generative AI that reads, understands, simplifies and translates the document.	Model: Amazon Nova 2 Lite (multimodal) via the global cross-region inference profile, Converse API, temperature 0.2.
Amazon Rekognition	Computer-vision gate: detects printed text before calling the AI model.	DetectText API reading the image directly from S3; at least 2 text lines required.
Amazon Polly	Neural text-to-speech for the explanation.	Neural engine; voices Kajal (English-IN and Hindi), Lucia, Lea, Vicki, Kazuha and Hala; MP3 output.
Amazon DynamoDB	Stores the history of processed documents.	Table vaanisetu-documents, on-demand capacity, key pk + createdAt for newest-first queries.
AWS Amplify Hosting	Hosts the static frontend over HTTPS with a global CDN.	App vaanisetu, branch main, deployed from a zip by the deploy script.
AWS IAM	Security: grants Lambda only the permissions it needs.	Role vaanisetu-lambda-role with a least-privilege inline policy (see Section 4.4).
Amazon CloudWatch	Monitoring: Lambda logs, errors and duration metrics.	Log group /aws/lambda/vaanisetu-api.

4. Code

The complete source code is in the project repository (<https://github.com/17mohak/vaanisetu>). It is organised into backend/ (Lambda), frontend/ (web app), infra/ (deployment script, IAM policy, sample generator) and report/. The most important parts are shown below.

4.1 Lambda: request routing

backend/lambda_function.py: lambda_handler

```
def lambda_handler(event, context):
    method = event.get("requestContext", {}).get("http", {}).get("method", "GET")
    path = event.get("rawPath", "/")
    if method == "OPTIONS":
        return respond(200, {})
    try:
        body = json.loads(event.get("body") or "{}") if method == "POST" else {}
        if path.endswith("/upload-url") and method == "POST":
            return create_upload_url(body)
        if path.endswith("/process") and method == "POST":
            return process_document(body)
        if path.endswith("/history") and method == "GET":
            return get_history()
        return respond(404, {"error": "Route not found"})
    except ValueError as e:
        return respond(400, {"error": str(e)})
    except Exception as e: # logged to CloudWatch
        print("ERROR", repr(e))
        return respond(500, {"error": "Processing failed. Please try again."})
```

4.2 Lambda: AI pipeline (Rekognition → Bedrock → Polly → DynamoDB)

backend/lambda_function.py: process_document

```
def process_document(body):
    key = body.get("key", "")
    lang = body.get("language", "en")
    if not re.fullmatch(r"uploads/[0-9a-f]{12}\\. (jpg|png)", key):
        raise ValueError("Invalid document key")
    if lang not in LANGUAGES:
        raise ValueError("Unsupported language")
    doc_id = key.split("/")[1].split(".")[0]
    timings = {}

    obj = s3.get_object(Bucket=BUCKET, Key=key)
    image_bytes = obj["Body"].read()
    if len(image_bytes) > MAX_IMAGE_BYTES:
        raise ValueError("Image is larger than 5 MB")

    # 1. Amazon Rekognition - verify the photo actually contains readable text
    t = time.time()
    detected = rekognition.detect_text(Image={"S3Object": {"Bucket": BUCKET, "Name": key}})
    lines = [d for d in detected["TextDetections"] if d["Type"] == "LINE"]
    timings["rekognition"] = round(time.time() - t, 2)
    if len(lines) < 2:
        raise ValueError("No readable text found. Please upload a clearer photo of a document.")
    ocr_confidence = round(sum(l["Confidence"] for l in lines) / len(lines), 1)

    # 2. Amazon Bedrock (Amazon Nova) - read, understand, simplify and translate
    t = time.time()
    analysis = analyse_with_bedrock(image_bytes, "png" if key.endswith("png") else "jpeg", lang)
    timings["bedrock"] = round(time.time() - t, 2)

    # 3. Amazon Polly - convert the simplified explanation into speech
    t = time.time()
    voice = LANGUAGES[lang]
```

```

speech = polly.synthesize_speech(
    Text=analysis["spoken_summary"][:2900], OutputFormat="mp3", Engine="neural",
    VoiceId=voice["voice"], LanguageCode=voice["code"],
)
audio_key = f"audio/{doc_id}-{lang}.mp3"
s3.put_object(Bucket=BUCKET, Key=audio_key, Body=speech["AudioStream"].read(),
    ContentType="audio/mpeg")
timings["polly"] = round(time.time() - t, 2)

# 4. Amazon DynamoDB - persist the result
record = {
    "pk": "DOC",
    "createdAt": int(time.time() * 1000),
    "docId": doc_id,
    "imageKey": key,
    "audioKey": audio_key,
    "language": lang,
    "documentType": analysis["document_type"],
    "title": analysis["title"],
    "sourceLanguage": analysis["source_language"],
    "extractedText": analysis["extracted_text"],
    "summary": analysis["spoken_summary"],
    "keyPoints": analysis["key_points"],
    "actionItems": analysis["action_items"],
    "ocrLines": len(lines),
    "ocrConfidence": Decimal(str(ocr_confidence)),
    "timings": {k: Decimal(str(v)) for k, v in timings.items()},
}
table.put_item(Item=record)
return respond(200, with_urls(record))

```

4.3 Lambda: Amazon Bedrock prompt and call

backend/lambda_function.py: prompt and analyse_with_bedrock

```

PROMPT = """You are VaaniSetu, an assistant that helps visually impaired and low-
literacy people understand documents.
Look at the photographed document and reply with ONLY a JSON object (no markdown) with
these keys:
"document_type": short category in English (e.g. "Medical Prescription", "Electricity
Bill", "Government Notice", "Bank Letter", "Exam Timetable", "Form", "Other"),
"title": a short title for the document in English (max 8 words),
"source_language": the language the document is written in (English name),
"extracted_text": the full text of the document, transcribed faithfully, preserving line
breaks,
"spoken_summary": a warm, simple explanation of what the document says and what the
reader must do, written in {language} for being READ ALOUD (plain sentences, no bullet
symbols, no markdown, under 900 characters; say amounts and dates naturally, and read
phone or ID numbers digit by digit),
"key_points": array of 3-5 short key facts in {language}, written with normal digits
(e.g. 2,170.50, 10 October 2026),
"action_items": array of 0-3 things the reader must do (with deadlines if any) in
{language}, written with normal digits.
Never invent information that is not in the document."""

def analyse_with_bedrock(image_bytes, fmt, lang):
    response = bedrock.converse(
        modelId=MODEL_ID,
        messages=[{"role": "user", "content": [
            {"image": {"format": fmt, "source": {"bytes": image_bytes}}},
            {"text": PROMPT.replace("{language}", LANGUAGES[lang]["name"])},
        ]}],
        inferenceConfig={"maxTokens": 3000, "temperature": 0.2},
    )
    text = response["output"]["message"]["content"][0]["text"]
    match = re.search(r"\{.*\}", text, re.S)
    data = json.loads(match.group(0) if match else text)
    for field in ("document_type", "title", "source_language", "extracted_text",
        "spoken_summary"):
        data[field] = str(data.get(field) or "")
    data["key_points"] = [str(x) for x in data.get("key_points") or []][:5]
    data["action_items"] = [str(x) for x in data.get("action_items") or []][:3]
    if not data["spoken_summary"]:
        raise ValueError("Could not understand this document")
    return data

```

4.4 IAM least-privilege policy

infra/lambda-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "MediaBucket", "Effect": "Allow", "Action": ["s3:GetObject",
"s3:PutObject"],
      "Resource": "arn:aws:s3:::__MEDIA__/*"},
    {
      "Sid": "DocumentsTable", "Effect": "Allow", "Action": ["dynamodb:PutItem",
"dynamodb:Query"],
      "Resource": "arn:aws:dynamodb:ap-south-1:__ACCOUNT__:table/__TABLE__"},
    {
      "Sid": "TextDetection", "Effect": "Allow", "Action": "rekognition:DetectText",
      "Resource": "*"},
    {
      "Sid": "SpeechSynthesis", "Effect": "Allow", "Action": "polly:SynthesizeSpeech",
      "Resource": "*"},
    {
      "Sid": "NovaOnBedrock", "Effect": "Allow", "Action": "bedrock:InvokeModel",
      "Resource": ["arn:aws:bedrock:*:__ACCOUNT__:inference-profile/global.amazon.nova-2-
lite*",
                    "arn:aws:bedrock:*::foundation-model/amazon.nova-2-lite*"]}
  ]
}
```

4.5 Frontend: upload and processing

frontend/app.js: direct-to-S3 upload and pipeline call

```
$("#go").addEventListener("click", async () => {
  if (!imageBlob) return;
  hideError();
  $("#go").disabled = true;
  $("#result").hidden = true;
  $("#empty").hidden = false;
  $("#empty").classList.add("busy");
  $("#empty").querySelector("p").textContent = "Reading your document...";
  let ticker;
  try {
    setStep("upload");
    const up = await api("/upload-url", { contentType: "image/jpeg" });
    const put = await fetch(up.uploadUrl, { method: "PUT", headers: { "Content-Type":
"image/jpeg" }, body: imageBlob });
    if (!put.ok) throw new Error("Upload to S3 failed");

    // The server runs the remaining stages in one call; animate them while we wait.
    setStep("rekognition");
    ticker = [setTimeout(() => setStep("bedrock"), 1200), setTimeout(() =>
setStep("polly"), 9000)];
    const result = await api("/process", { key: up.key, language: selectedLang });
    ticker.forEach(clearTimeout);
    setStep("all");
    showResult(result, true);
    loadHistory();
  } catch (err) {
    (ticker || []).forEach(clearTimeout);
    setStep("none");
    showError(err.message);
    $("#empty").querySelector("p").textContent = "Your explanation will appear and play
here.";
  } finally {
    $("#go").disabled = false;
    $("#empty").classList.remove("busy");
  }
});
```

4.6 Deployment script (excerpt)

infra/deploy.sh: creating the HTTP API and wiring it to Lambda

```

echo "== API Gateway (HTTP API)"
API_ID=$(aws apigatewayv2 get-apis --query "Items[?Name=='vaanisetu-api'].ApiId" --
output text)
if [ -z "$API_ID" ] || [ "$API_ID" = "None" ]; then
    API_ID=$(aws apigatewayv2 create-api --name vaanisetu-api --protocol-type HTTP \
--cors-configuration
'AllowOrigins=*,AllowMethods=GET,POST,OPTIONS,AllowHeaders=content-type' \
--query ApiId --output text)
    INT=$(aws apigatewayv2 create-integration --api-id $API_ID --integration-type
AWS_PROXY \
--integration-uri "$FN_ARN" --payload-format-version 2.0 --timeout-in-millis 29000 \
--query IntegrationId --output text)
    for r in "POST /upload-url" "POST /process" "GET /history"; do
        aws apigatewayv2 create-route --api-id $API_ID --route-key "$r" --target
"integrations/$INT" >/dev/null
    done
    aws apigatewayv2 create-stage --api-id $API_ID --stage-name '$default' --auto-deploy \
--default-route-settings ThrottlingBurstLimit=20,ThrottlingRateLimit=10 >/dev/null
    aws lambda add-permission --function-name $FN --statement-id apigw-invoke \
--action lambda:InvokeFunction --principal apigateway.amazonaws.com \
--source-arn "arn:aws:execute-api:$AWS_REGION:$ACCOUNT:$API_ID/*" >/dev/null
fi

```

5. Screenshots

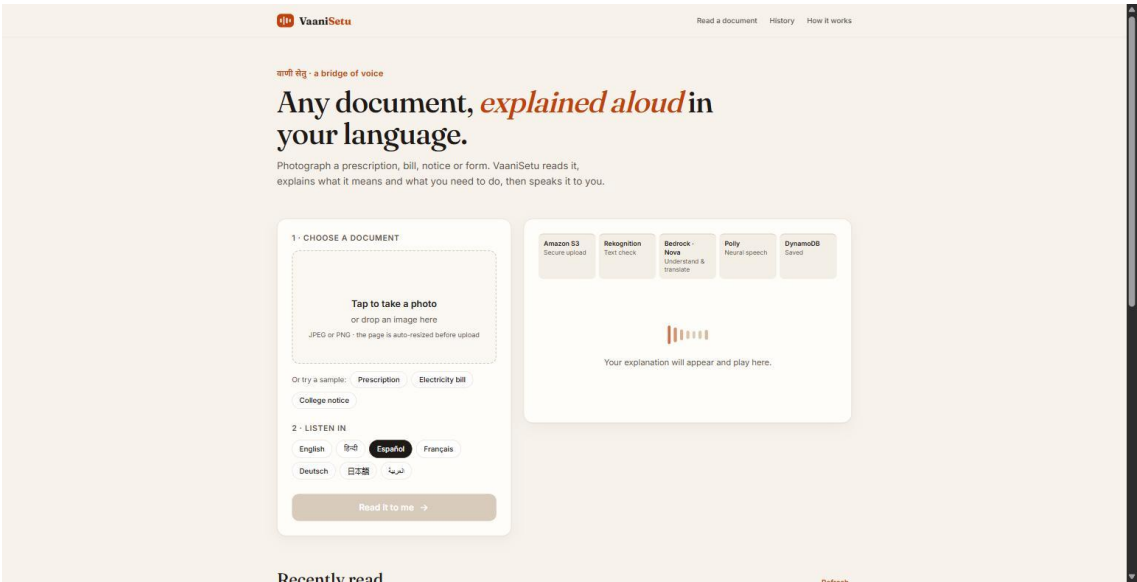


Figure 5.1: VaaniSetu home page (hosted on AWS Amplify)

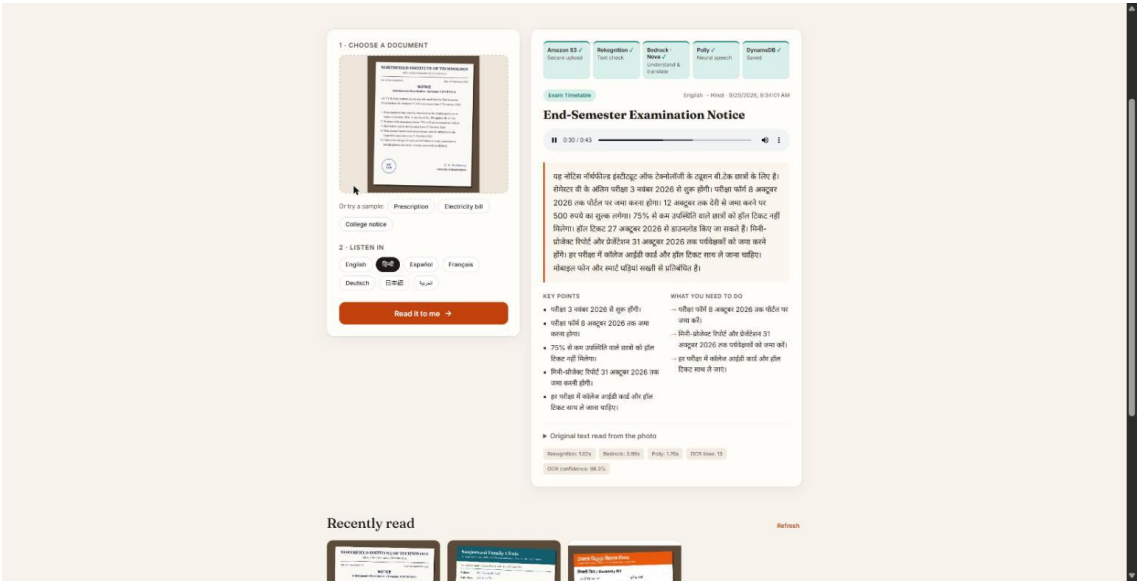


Figure 5.2: English college notice explained and spoken in Hindi; the pipeline tracker shows every AWS service completed

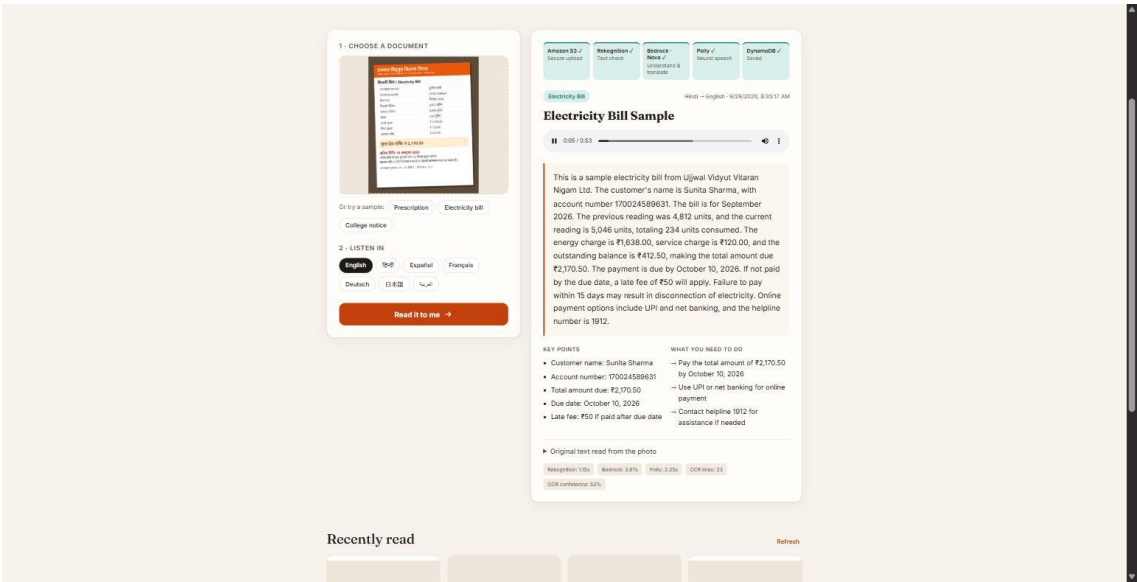


Figure 5.3: Hindi electricity bill explained in English, with key points, action items and per-service timings

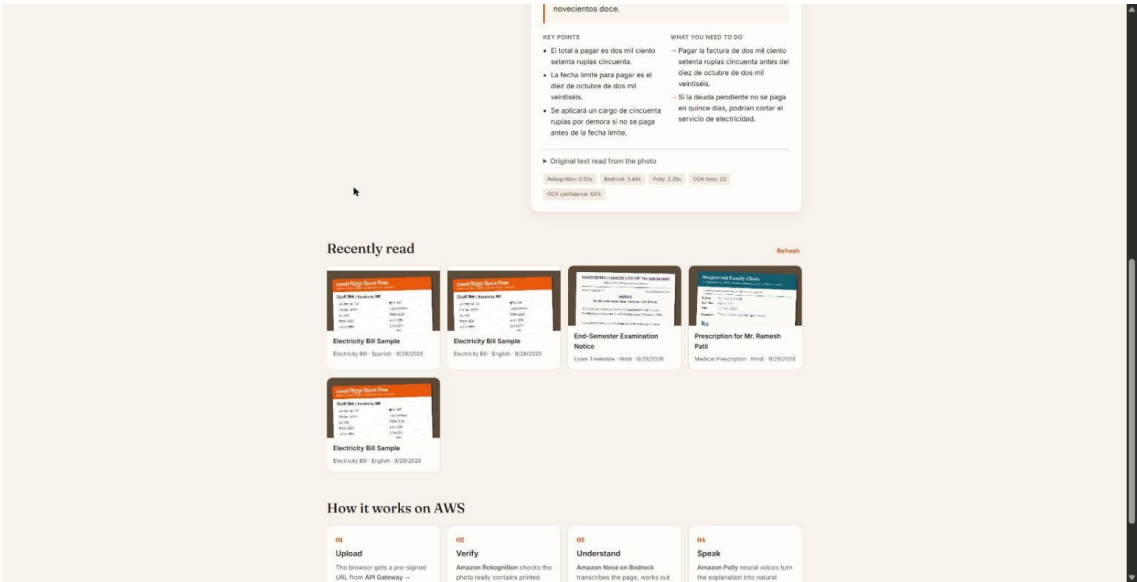


Figure 5.4: History of processed documents loaded from Amazon DynamoDB

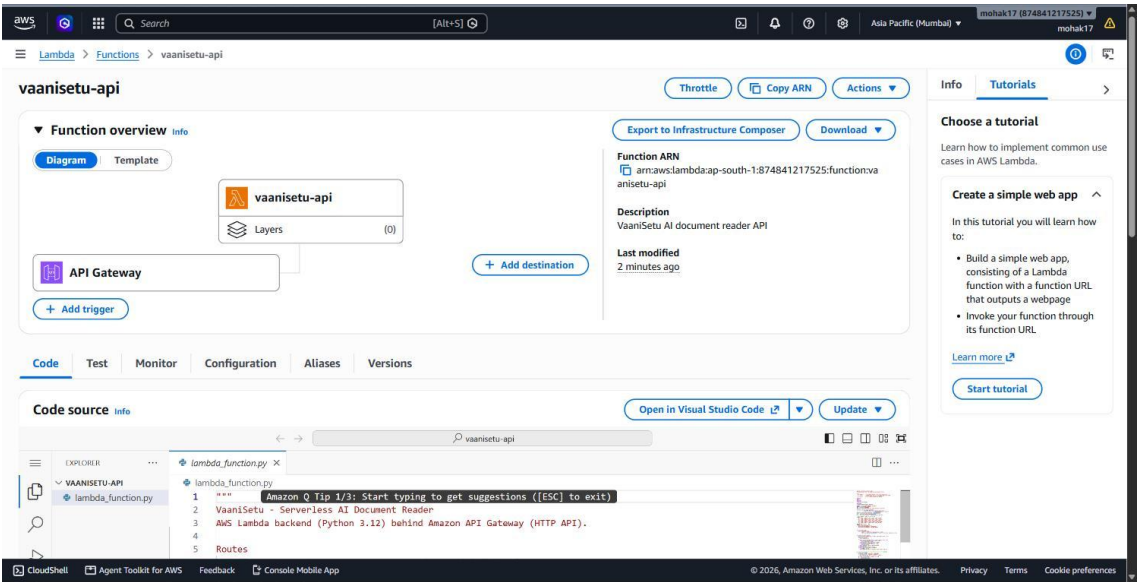


Figure 5.5: AWS Lambda function *vaanisetu-api* with its API Gateway trigger

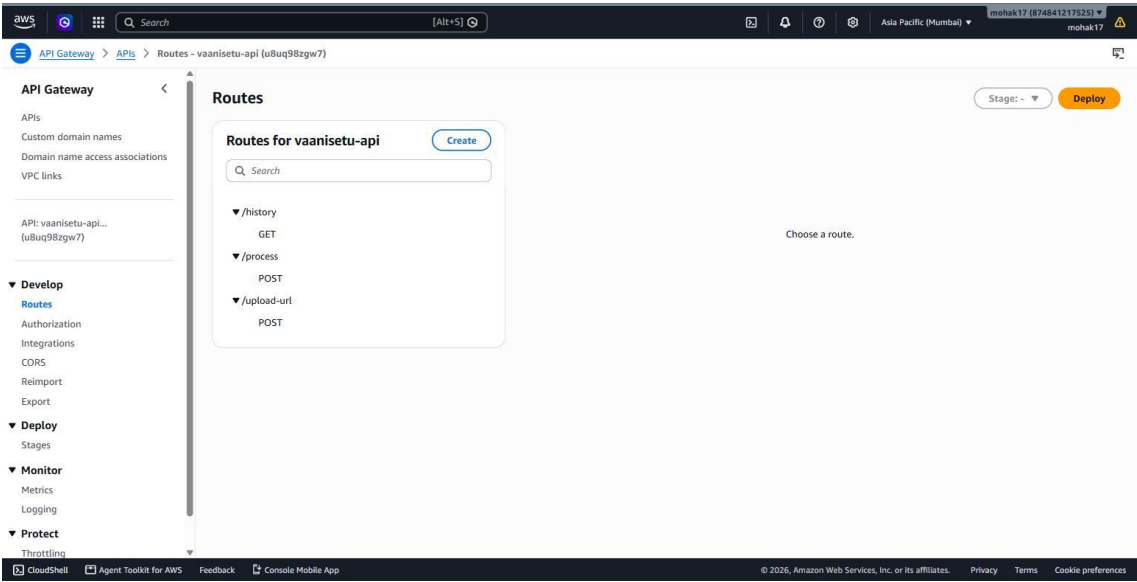


Figure 5.6: Amazon API Gateway HTTP API routes

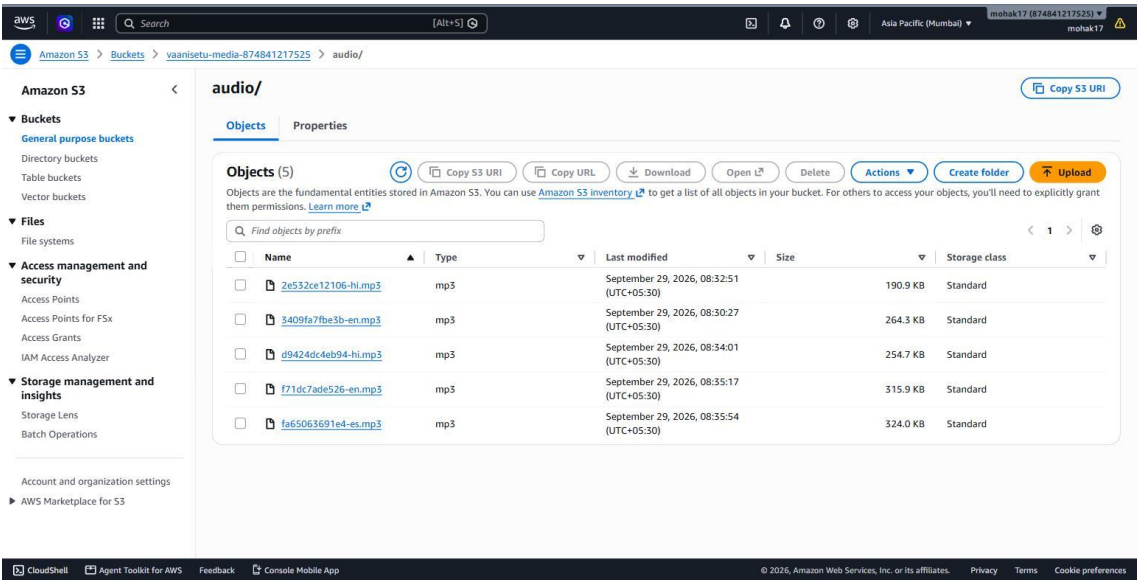


Figure 5.7: Amazon S3 bucket with MP3 files generated by Amazon Polly

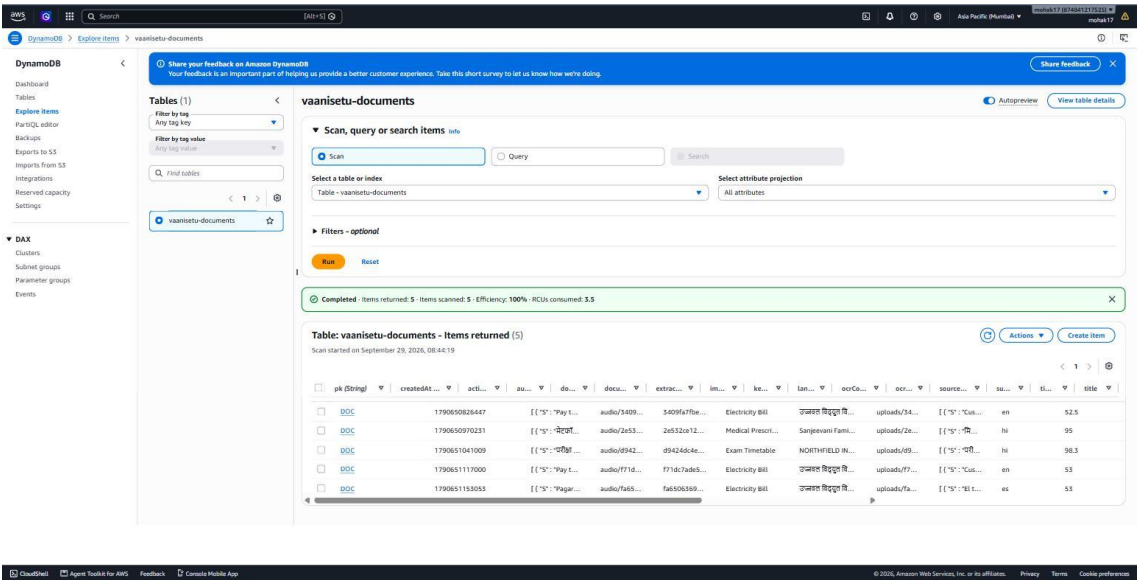


Figure 5.8: Amazon DynamoDB table vaanisetu-documents with stored results

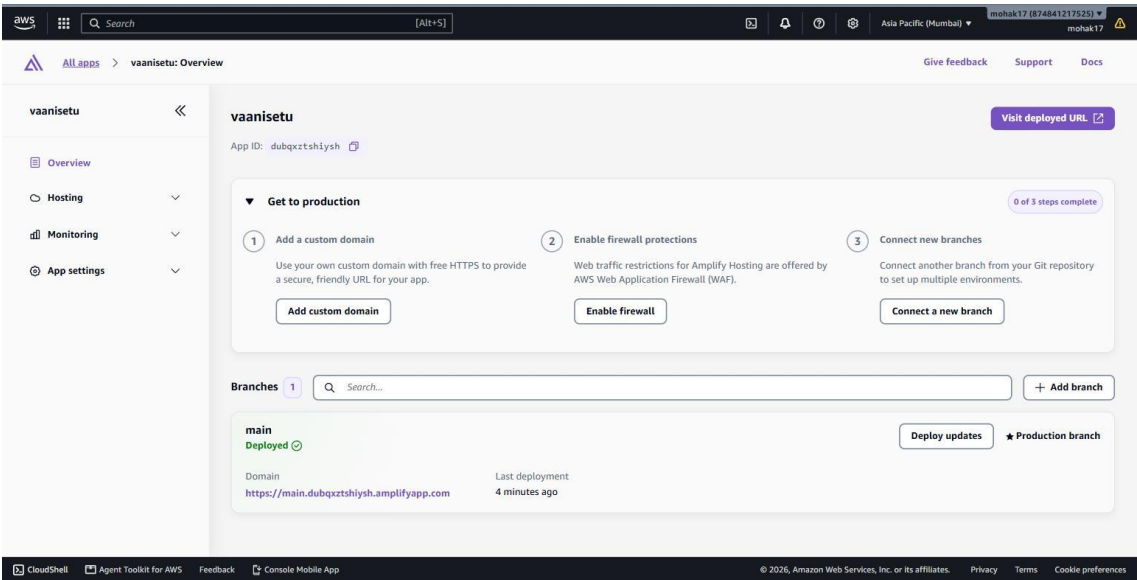


Figure 5.9: AWS Amplify Hosting app with the deployed main branch

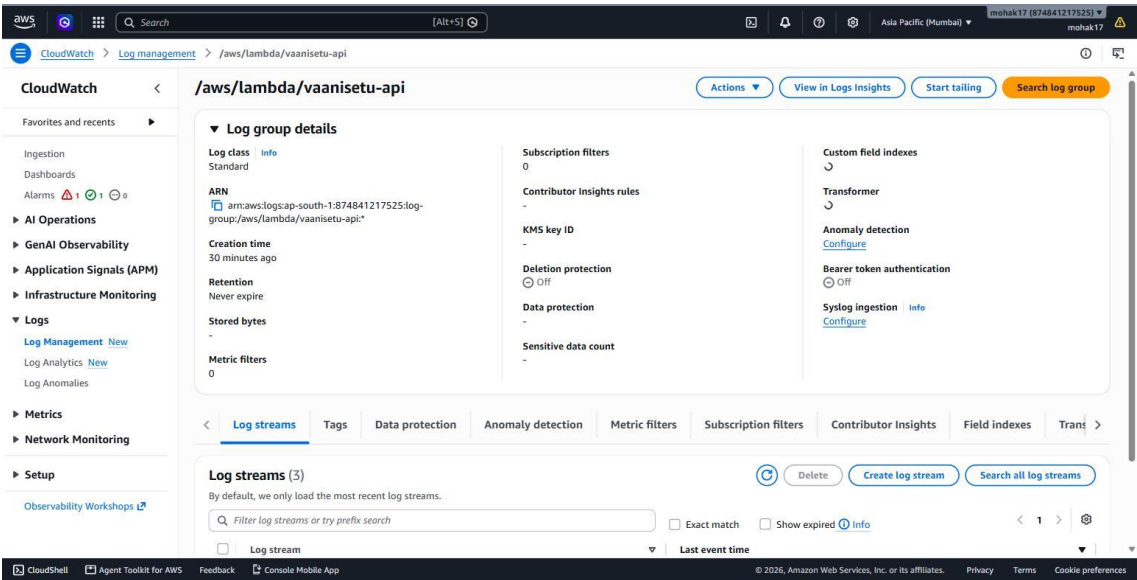


Figure 5.10: Amazon CloudWatch log group for the Lambda function

6. Conclusion

VaaniSetu shows how managed cloud services can be combined into a useful, socially relevant product with very little infrastructure code. A photo taken on a phone becomes a clear spoken explanation in the user's own language in about ten seconds. In testing, Amazon Rekognition took about 1 second, Amazon Bedrock (Nova 2 Lite) about 4 seconds and Amazon Polly about 1.5 to 2 seconds per document.

6.1 Outcomes

- A fully serverless application on AWS integrating eight core services plus IAM and CloudWatch.
- Documents in one language (for example Hindi) explained and spoken in another (for example English).
- Security by design: private S3 buckets, short-lived pre-signed URLs, encryption at rest and a least-privilege IAM role.
- Pay-per-use cost: there are no running servers, so the application costs nothing when idle.
- Infrastructure provisioned by a single repeatable script, with source code under version control.

6.2 Challenges and Learnings

- The AWS Free-plan account did not allow Amazon Textract, Translate or Comprehend. The design was changed to use a multimodal model on Amazon Bedrock, which reads, translates and summarises in a single call.
- Amazon CloudFront required account verification for new accounts, so the frontend was hosted on AWS Amplify Hosting, which also provides HTTPS and a CDN.
- Rekognition's text detection supports Latin script only. It is therefore used as a fast, low-cost "is there text?" gate, while Bedrock performs the multilingual reading.
- API Gateway limits requests to 29 seconds, so uploads go directly to S3 and the AI pipeline was designed to finish in about 10 seconds.

6.3 Future Scope

- Support multi-page PDFs using asynchronous processing with Amazon SQS or AWS Step Functions.
- Add more Indian languages as Amazon Polly voices become available, and a voice question-and-answer mode ("When is my next dose?").
- User accounts with Amazon Cognito so that each user's history is private.
- A WhatsApp or IVR channel so that users without smartphones can send a photo and receive a call back.

Live application: <https://main.dubqxztshiysh.amplifyapp.com>

Source code: <https://github.com/17mohak/vaanisetu>